

Data management Assignment 1

Hektor Dahlberg (2201899)

May 8, 2025

Contents

1	Formulating the business case	4
1.1	The advantages of a unified data management system	6
1.2	the types of data essential for operational and strategic decisions .	7
1.3	Key steps to transition to an integrated database system using a systems development life cycle approach.	8
2	Conceptual design	9
2.1	Task 1: Additional entities etc.	9
2.2	Create an ER-diagram	11
2.3	Ensure that your final data model addresses these considerations: .	12
3	Logical design	14
3.1	Relational and non-relational databases	14
3.2	Relational schema	15
3.3	Evaluate the current normalization of your design	18
4	Physical database design and performance	18
4.1	Denormalization	18
4.2	Usage analysis based on LearnHubAI's operational size.	19
5	Data Warehousing and data integration	20
5.1	Benefits of EDW compared to independent data marts	20
5.2	Identify attributes that are most likely to require manual input by staff or instructors	21
6	Exploring the Role of LLMs in Data Management	21

6.1	In what ways can LLMs support data management activities? . . .	21
6.2	What are the potential benefits of using LLMs for Learn-HubAI? .	22
6.3	What are the challenges and risks, including issues related to accuracy, cost, explainability, or integration with existing systems? .	22
6.4	Would it be realistic or beneficial to integrate LLM-based components into any part of that architecture?	24
7	Final comment	24

1 Formulating the business case

Effective data management is essential not only for understanding student behavior but also for making good and informed decisions. Right now the data-management is an fragmented data system where teams collect data independently using spreadsheets, isolated tools and separate cloud-based systems, and as such the decision-making is based on incomplete or delayed information. There is no centralized database, which hinders collaboration between the different teams and makes in difficult to get a comprehensive view of overall business performance. Also, there is no standardized protocol on how the data is stored, what data is stored, most likely duplicates of information and inconsistencies in data.

The current system has at least these problems:

- Data silos.

Data are maintained by different teams with no protocol on how the information is shared with the other teams.

- Inconsistency in data

Different teams use different methods to store the data, as such most likely the type of data that is stored is not consistent. As the naming conventions, formats and tools use differ it is difficult to unify the data into one.

- Manual updates/ Delayed decision making

Some teams rely on spreadsheets, which most likely needs to be manually updated, and as such can make the data used for different decision making by different teams totally different if the data is not up to date.

- System safety

No information on where the data is gathered other than how it is, for example having users information in plain text can be a safety concern, and most likely the user data is not deleted after it is not needed so goes against the GDPR.

As LearnHubAI has a growing user base, it is important to get data management up to date, as the current data management system will lead to problems in the future and will cost a lot more to fix when the user base grows.

Constraints/considerations for changing the data management system:

- Limited technical staff

LearnHubAI seems not to have any specialized IT personnel. As such the solution that will be adopted needs to take this to account.

- Modular approach

Given that the different teams manage their data separately, the system can be developed and deployed on a set of time-boxed iterations which minimizes the disruption that the in their day to day normal operations.

- Cloud based infrastructure

Rather than managing a own IT infrastructure, LearnHubAI could choose to adopting a cloud-based approach, such as Infrastructure-as-a-service or Platform-as-a-service. These models would offer a a scalable, low upfront cost and minimize the upkeep of the system.

- Build vs Buy

Building an custom system is a more expensive option, but it can be more flexible and tailored to the specific needs, while an off-the-shelf solution would be a cheaper option but it is not as flexible, if there is an off-the-shelf solution which fulfills the needs of LearnHubAI's needs then an off-the-shelf solution could be the way to go.

LearnHubAI has identified three strategic priorities:

1. Improve student engagement through better tracking of activity and performance.
2. Streamlining content management by integrating instructor and course data.
3. Enhancing operational reporting across departments, especially in marketing and customer support.

These three priorities will serve as a guideline for designing and implementing the data management system. Any decision regarding the structure, access and governance of the database should and will ensure that all the teams can have access to relevant data effectively and securely.

1.1 The advantages of a unified data management system

A unified data management system would offer LearnHubAI a centralized, consistent and scalable way to handle their data across all the teams. It would solve the problems which were talked about before (data silos, data inconsistency, delayed decision making, system safety). Additionally, a unified system would simplify

compliance with GDPR and other data protection regulations and remove the reliance on manual updating/processes which would make the company also more efficient in its operations.

1.2 the types of data essential for operational and strategic decisions

- User activity data

Track the behavior of the users of the site, such as logins, time spent on lessons, click paths etc. This data is important to fulfill priority 1 from the list that LearnHubAI had identified

- Instructor data

Teaching history, qualifications, feedback etc. Helps with scheduling the courses for the instructors and makes sure that the instructor is qualified to teach the course.

- Enrollment patterns

Course registration, completion rates, drop out rates etc. Helps forecasting demands, evaluation of course popularity and planning for future courses.

- Content rating and feedback

Gives information to the instructors and the company on how well a course/instructor was received and what should be improved in the future.

- Platform usage metrics

Server logs, error reports, uptime stats, peak usage periods, access points and load time. Makes sure that the page runs smoothly and helps when deciding on server capacity and load balancing of the servers at the same time gives information if anything would happen that the error can be pinpointed a bit more easily.

1.3 Key steps to transition to an integrated database system using a systems development life cycle approach.

1. Planning & requirement analysis

Understand and strategic goals/ business goals of LearnHubAI, assess how their current data management system is working, planning of the resources and finding out the requirements from LearnHubAI for the system.

2. Defining requirements

From the information gathered in step 1, talk to the different teams on their needs and user roles, and how they expect the system to work and defined these requirements and rank them, other requirements would be data types, GDPR compliance etc.

3. Design

Design the database schema, system architecture and plan on integration of existing tools and data to the new system, also to consider is to ensure that the system will be easily scalable for future use.

4. Development

Develop it.

5. Testing

Test that everything works as it should, monitor performance and get feedback from end users via test runs.

6. Deployment & Maintenance

Deploying it and maintenance most likely some bugs or problems were missed during the testing period which will be addressed during this period.

2 Conceptual design

2.1 Task 1: Additional entities etc.

Entities like MarketingCampaign and SupportTicket could be added to track campaign effectiveness and gather data on how quickly student support needs are addressed. However, to keep the focus of this assignment on the main task, I chose not to include these elements in the ER diagram to avoid overengineering.

Additionally, elements like how payments would be handled could also be implemented, but this falls outside the scope of this task. To design that properly, more information about how LearnHubAI operates would be needed. Moreover, payment data would likely require a separate database dedicated to financial information.

Since a course can either be live or self-paced, I added a CourseType attribute to the main Course entity and created two sub-entities: LiveCourseDetail and SelfPacedCourseDetail. These subtypes are explained in more detail in the ER diagram. I also included a Price attribute in both subtypes, as pricing usually differs between courses and is an important part of their structure. Additionally,

I added a Version attribute to allow for different numbering or versions of the course if necessary.

For Instructors, I included Type and Status attributes. This allows the system to differentiate between full-time instructors and substitutes, as well as track whether an instructor is currently active or on leave. These additions are important for better course planning and resource management.

To make the Assessment entity reusable across multiple courses, I created a junction table called CourseAssessment. This allows an assessment to be used in many courses, making it easier for LearnHubAI to build a bank of reusable assessments that can be linked to different courses as needed.

This approach not only streamlines the database but also ensures flexibility and scalability as LearnHubAI grows. It follows the company's three strategic priorities: improving student engagement by better tracking of student activity through login data, integrating instructor and course information to enhance content management, and supporting customer service by incorporating content feedback for continuous improvement.

2.2 Create an ER-diagram

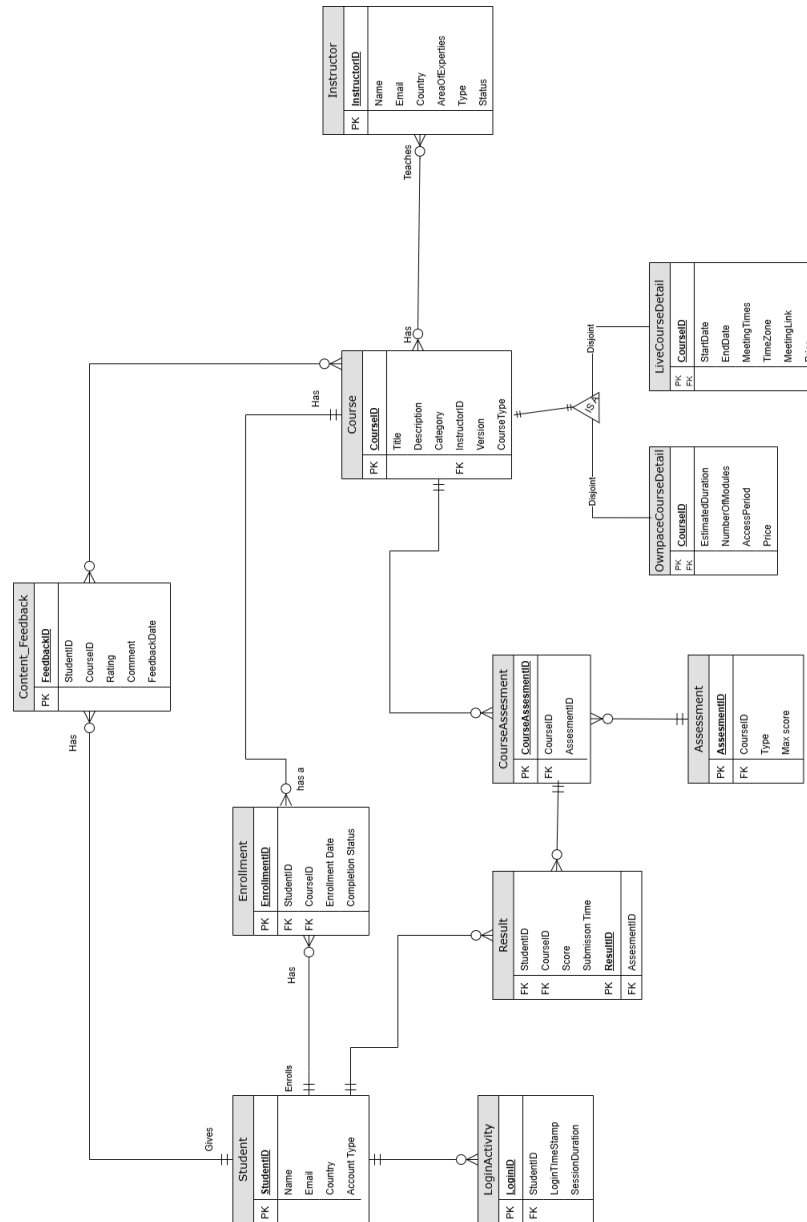


Figure 1: ER Diagram

2.3 Ensure that your final data model addresses these considerations:

The first draft of the ER diagram had some flaws, which I addressed in the final version (Figure 2) by making the following changes:

1. Accommodating updated courses without losing historical data

I introduced a CourseVersion entity to allow courses to have multiple versions over time, while preserving the history of course updates. This ensures that the system tracks version history without losing previous course data.

2. Capturing student engagement and session duration

I added the ContentAccessed and LogoutTimeStamp attributes to the Login Activity entity. This ensures we get data from students on what data they access and when, which helps us to get an insight on student interaction while visiting the site.

3. Including essential assessment data and feedback ratings

I created the AssessmentFeedback entity to allow students to provide feedback on specific assessments.

4. Supporting instructor performance review and content improvement processes

Did not change anything as ContentFeedback already captures both feedback on the course and the instructor.

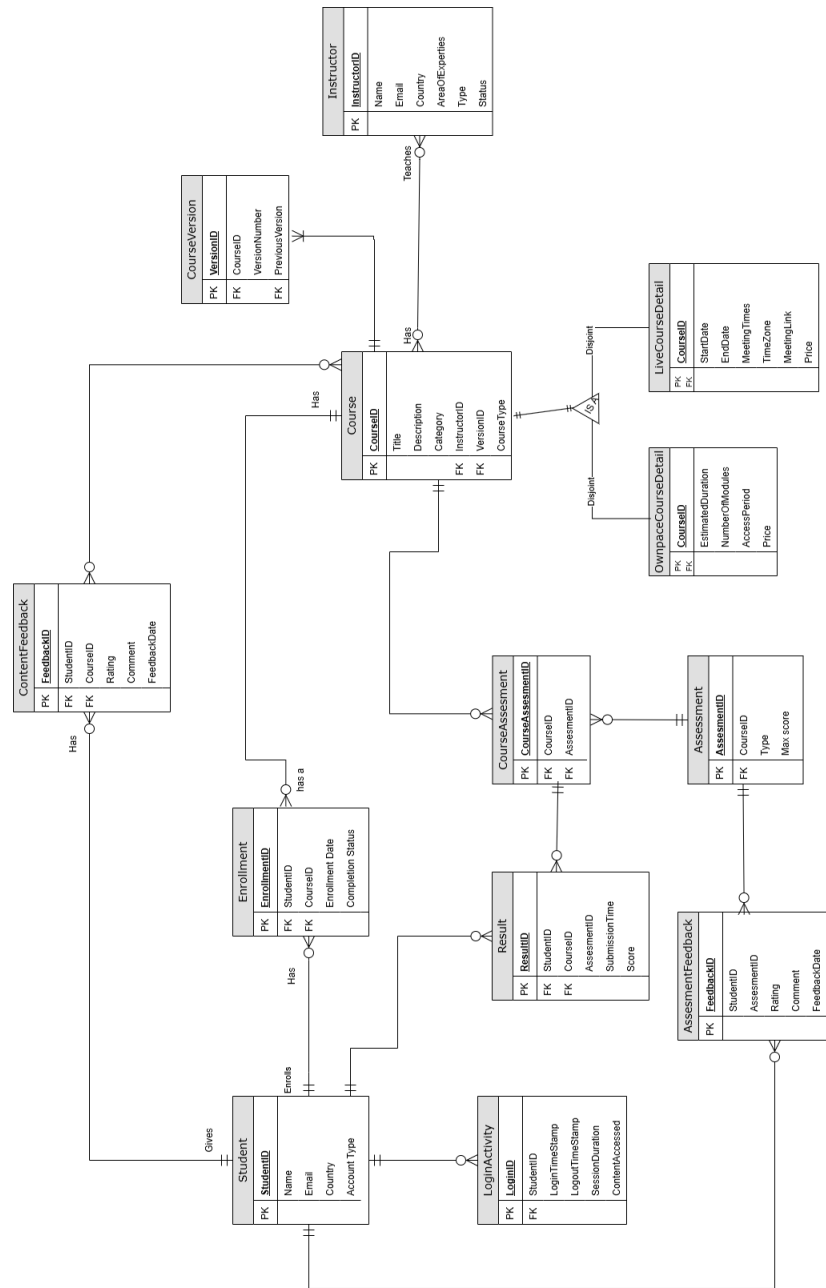


Figure 2: Final ER Diagram

3 Logical design

3.1 Relational and non-relational databases

Relational databases have the strong advantage of providing a structured, consistent, and reliable way to store data using well-designed schemas. However, this structure can also be a weakness, as it offers little to no flexibility. Relational databases also support ACID properties (Atomicity, Consistency, Isolation, Durability), which are essential for ensuring data integrity and reliability—especially in critical applications like banking.

On the other hand, non-relational databases are ideal for handling unstructured data such as multimedia files and session logs. NoSQL databases tend to be more scalable and better suited for real-time applications. In the case of LearnHubAI, storing content-related data, assessment feedback, and session tracking in a non-relational database would likely be more effective, especially considering the large volume of log data generated. From personal experience, setting up a non-relational database is also “easier” since there’s less complexity involved during the initial setup.

So, should LearnHubAI consider a hybrid architecture? In my opinion, yes. Core structured data such as students, enrollments, and courses should still be stored in a relational database. However, using a non-relational database for logs and feedback would be a smart choice. These systems handle unstructured data more efficiently, are more cost-effective, and are better suited for real-time analytics, which are crucial for scaling decisions and maintaining the platform effectively.

3.2 Relational schema

Student

StudentID(PK), Name, Email, Country, AccountType

LoginActivity

LoginID(PK), StudentID(FK), LoginTimeStamp, LogoutTimeStamp, SessionDuration (derived), ContentAccessed

AssesmentFeedback

FeedbackID (PK), StudentID (FK1), AssessmentID (FK2), Rating, Comment, FeedbackDate

Result

ResultID(PK), StudentID(FK1), CourseID(FK2), AssesmentID, SubmissionTime, Score

Enrollment

EnrollmentID(PK), StudentID(FK1), CourseID(FK2), EnrollmentDate, CompletionStatus

Assesment

AssesmentID(PK), CourseID(FK), Type, MaxScore

CourseAssesment

CourseAssesmentID(PK), CourseID(FK1), AssesmentID(FK2)

ContentFeedback

FeedbackID(PK), StudentID(FK1), CourseID(FK2), Rating, Comment, FeedbackDate

Course

CourseID(PK), Title, Description, Category, InstructorID(FK1), VersionID(FK2), CourseType

Instructor

InstructorID(PK), Name, Email, Country, AreaOfExperties, Type, Status

CourseVersion

VersionID(PK), CourseID(FK1), PerviousVersion(FK), VersionNumber

OwnPaceCourseDetail

CourseID(PK,FK), EstimatedDuration, NumberOfModules, AccessPeriod, Price

LiveCourseDetail

CourseID(PK,FK), StartDate, EndDate, MeetingTimes, TimeZone, MeetingLink,
Price

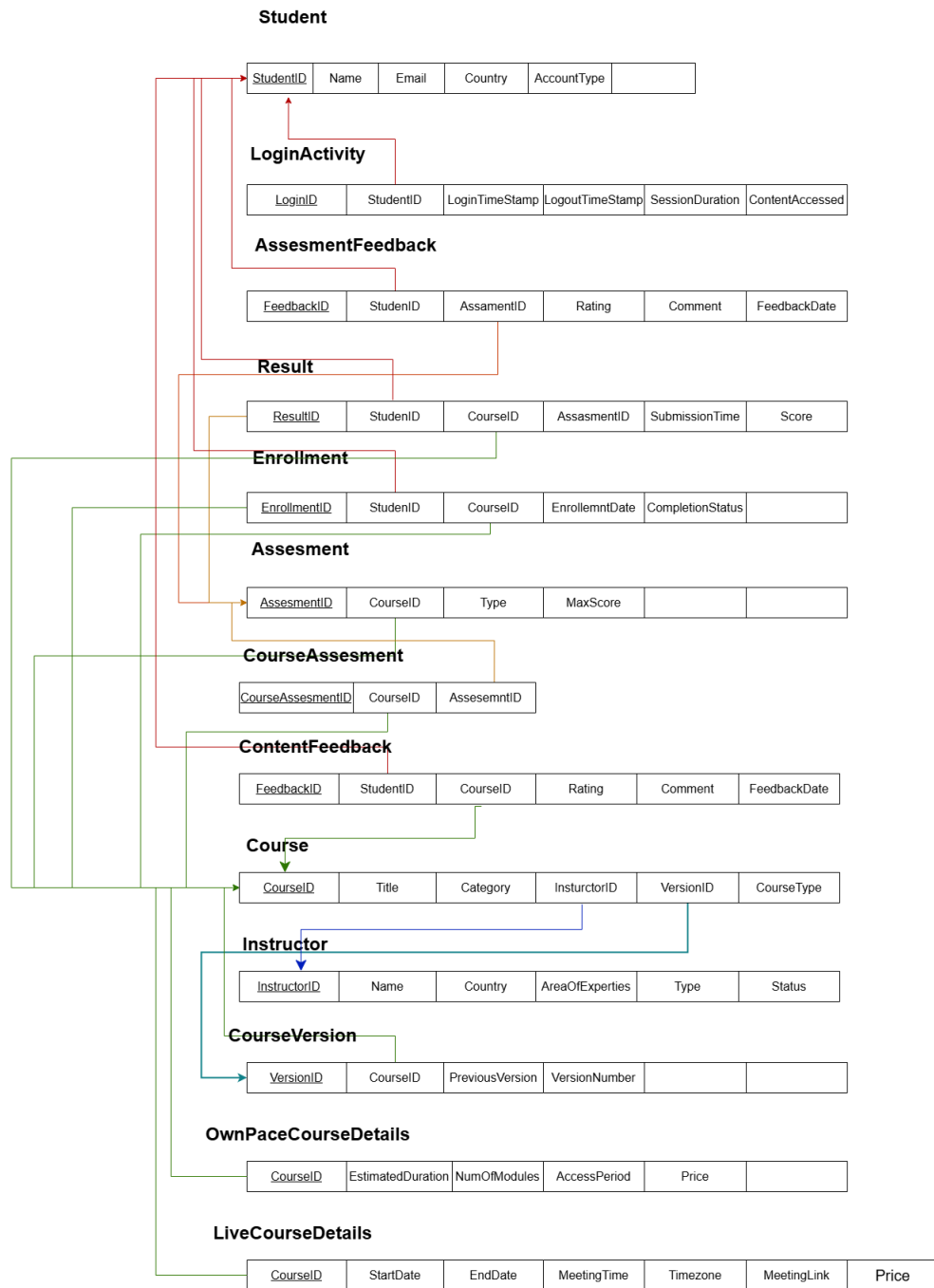


Figure 3: Relational Schema

3.3 Evaluate the current normalization of your design

First Normal Form: Is satisfied as all attributes have atomic values and each table has a clear primary key.

Second Normal Form: Is satisfied as there are no composite keys.

Third Normal Form: Is satisfied for all except SessionDuration as it is depends on the LoginTimeStamp and LogoutTimeStamp, all other are non-key attributes that only depend on the primary key and there are no transitive dependencies. To make it adhere with third normal form, I would just remove the SessionDuration and instead calculate it via a query when needed.

4 Physical database design and performance

4.1 Denormalization

Denormalization is a database optimization technique in which we add redundant data to one or more tables. The point of denormalization is to make queries execute quickly, so removing costly joins and other operations which would take time. Normally this is more important in real time systems where time is of making the query is important. Example for LearHubAI is that for finding the name/names of instructors for a course we must make a query to join the course table with the instructor table. This can be fine but if the data is large and the query is often used then we are spending a lot of time doing joins on tables. The idea with denormalization is then to make all this to one table, the drawback is that we will have redundant data in our table, this redundant data comes as a cost of increased storage requirements which leads to increased costs of maintaining the database. So in

short denormalization is technique where you decrease the numbers of tables to make your queries quicker with a cost of redundant data.

Where denormalization could be used other than finding the name of the teacher in LearHubAI is for a student progress tracking, so make a summery-table called StudentCourseProgress which could be a join of assessment, result and Enrollment. Another could be for InstructorDashboard which would be a joining of instructor, course, contentfeedback, with the attributes, instructorID, CourseCount, TotalEnrollments, AvgFeedback etc.

4.2 Usage analysis based on LearnHubAI's operational size.

1. Course progress tracking

50 000 students * 3 courses = 150 000 enrollments

150 000 enrollments * 5 assessments = 750 000

750 000 * 2(once to give the assessment and once to read it by the student)
= 1 500 000 queries

So more than 1 500 000 queries per year. Where at least 2 joins are needed (Student, Assessment, Result).

Benefit with denormalization is that it allows instant lookup without having to do 2 joins to get the results.

2. Instructor performance reporting

50 000 students * 3 courses = 150 000 enrollments

150 000 course feedbacks per year

Where at least 3 joins are needed (Instructor, Course , Enrollment, Content-Feedback)

Benefit with denormalization is it allows instant lookup without having to do 3 joins to get the results.

3. Engagement pattern analysis

$50\,000 \text{ students} * 1 \text{ login per day} * 365 = 18\,250\,000 \text{ login activates}$

Benefit with denormalization is it avoids making repeated joins on over 18 000 000 login records.

5 Data Warehousing and data integration

5.1 Benefits of EDW compared to independent data marts

The benefit with Enterprise Data warehouses is that they centralize the data from multiple data source into a unified structure, this makes it easier to analyse the data and get a clearer picture of the overall organization. While a data mart setup has multiple siloes sources of information, an EDW provides a consistent view of the data across all teams. This consistency gives the possibility for more accurate reporting, better decision-making, and improved collaboration across teams.

Is near real-time data integration necessary for decision-making for LearnHubAI?

For LearnHubAI, near real-time data integration is most likely not necessary for most of the system, as most of its core operations can be handled on a periodic processing where time is not that important. A real-time data integration for the whole system would be overkill in my opinion, but a real-time data integration for operational monitoring like user behaviour. This could both help with detecting unusual login patterns, like someone trying to brute-force users login details,

shared login credentials, spamming etc. With real-time data integration an intervention could be triggered to freeze the accounts and alert the student that their account is compromised, also would trigger an alert for admins that there is something suspicious going on, which would help to make sure that user data is not getting compromised. But in general, near real-time integration is not essential for analytics on students progression and other user data which is not a security risk.

5.2 Identify attributes that are most likely to require manual input by staff or instructors

and normally require manual input. Same goes for instructors as they are not going created the same way as a user account is created, especially instructor status is going to be updated manually as it often involves an administrative decision to go on a leave. Depending on how the assessment of the grade is done, so if it is a test for example the results maxscore maybe manually evaluated by a teacher but can also be for example multiple question tests where input by instructors is not needed.

6 Exploring the Role of LLMs in Data Management

6.1 In what ways can LLMs support data management activities?

For query generation can help users without technical background to make the data accessible to them.

For data cleaning can help with typos and non-standard formats like the feedback. For example for feedback it can be used to standardize the text or even use natural language processing to make useful data for analytics.

For conceptual modelling it can help to improve databases and even generate the base on which to build your database on, normally do it gets 80% correct so still need a professional to make the final tweaks, but during the process it can give ideas on how it could be solved.

6.2 What are the potential benefits of using LLMs for LearnHubAI?

Because the user base for learnHubAI is quite small I would not recommend any bigger implementation of LLM, but if we consider that the company is expected to grow quite rapidly I would say the biggest benefit for using LLMs would be automation of tasks. Tasks like tagging of data, summarizing but also automatization of the sentiment analysis of feedback would be use cases the learnHubAI should consider.

6.3 What are the challenges and risks, including issues related to accuracy, cost, explainability, or integration with existing systems?

Accuracy: LLMs are quite a new thing so they are not totally discovered yet, meaning there are always challenges and risks when implementing the newest technology. The biggest problem in my opinion is still hallucinations which could lead to serious errors in data interpretation, as I said from personal experience

normally LLMs like chatGPT can do simple tasks quite well, but in harder tasks there is quite a lot of prompt engineering that needs to be done, and still after all that it can be that for example the code looks correct but when trying to run it or looking more closely you can see that it makes quite simple mistakes.

Cost: Building your own LLMs in this case is out of the question, as it is way too expensive. So for smaller companies like LearnHubAI, they typically opt to use LLMs as a service, such as through APIs provided by OpenAI. But still this can be quite expensive. OpenAI o3 costs \$40 /1M output tokens and \$30 /1M output tokens, A token being one word so "How many tokens?" would be "How" "many" "tokens" " ". This can pile up quite quickly even for a company of learn-HubAI's scale and lead to high costs, and even this all before we can talk about the integration into existing system, building the UI around the LLMs and maintenance.

Explainability: LLMs are black boxes, you know your inputs and get an output but you have no idea how it got to that solution. This lack of knowledge can be problematic as LLMs can have false positives and also false negatives. For example for Feedback it could interpret a positive feedback as negative as it didn't take into consideration clues of sarcasm or cultural context. This can lead to that the data which is used for decision making is flawed and as such the analysis of the data is also flawed.

Integration: Integration of LLMs to an existing structured database would require time and a lot of technical skill, as new middlewares/ data pipelines etc. would

be needed to be developed. There would be in the end an increased complexity to the database, which would require a lot of testing and modifications, which would lead to increased cost, especially for a small company like LearnHubAI.

6.4 Would it be realistic or beneficial to integrate LLM-based components into any part of that architecture?

In my opinion it would be realistic and beneficial but only on a small scale, for example automation of tagging/classification of the feedback, other than that I do not see a use case for LLMs for a company which provides courses for students and is not a big company.

7 Final comment

ChatGPT was used to clean up my text, and I verified that it reflected my own thoughts, including this sentence.